Staj haftasonu(10.gün)
RAG Evaluation (RAG Değerlendirme)

1.Bölüm
1. Test Seti Hazırlama (Curate a Test Set)

- **1.1 Neden Test Seti?**
- **1.2 Test Kaydı**
- **1.3 JSONL**
- **1.4 Pydantic**
- **1.5 Kategoriler**

2. Bölüm

Retrieval Evaluation (Retriever Değerlendirme)

- **2.1 Akış**
- **2.2 MRR**
- **2.3 nDCG**
- **2.4 Recall@K**
- **2.5 Precision@K**
- **2.6 Keyword Coverage**

3. Bölüm

Answer Evaluation ve RAG Optimization

- **3.1 Answer Evaluation**
- **3.2 LLM-as-a-Judge**
- **3.3 Pydantic Doğrulaması**
- **3.4 Kodların Mantığı**
- **3.5 RAG Optimization**
- **3.6 Günün Çıkarımı**

4.Bölüm
Advanced RAG Techniques

- **4.1 Amaç**
- **4.2 Chunking & Embedding**
- **4.3 Prompt & Preprocessing**
- **4.4 Query Rewriting & Expansion**
- **4.5 Re-Ranking**
- **4.6 Hierarchical & Graph RAG**
- **4.7 Agentic RAG**
- **4.8 RAG Öldü mü? / AI Engineer Bakışı**

5.Bölüm Advanced RAG Laboratuvarı

- **5.1 Amaç**
- **5.2 Belgeleri Yükleme**
- **5.3 Semantic Chunking**
- **5.4 Embedding ve Chroma'ya Kaydetme**
- **5.5 Vektörleri Görselleştirme**
- **5.6 Re-Ranking**
- **5.7 Query Rewriting**
- **5.8 Final RAG Pipeline**
- **5.9 AI Engineer Bakış Açısı**

6. Bölüm

RAG Sistemini Deneylerle Optimize Etme ve Değerlendirme

- **6.1 Genel Akış ve ingest.py**
- **6.2 Chunk, Splitter ve Embedding Deneyleri**
- **6.3 Embedding'in Cevap Kalitesine Etkisi**
- **6.4 Retrieval Evaluation**
- **6.5 Answer Evaluation**
- **6.6 Genel Sonuç ve AI Engineer Notu**

1.Bölüm
1. Test Seti Hazırlama (Curate a Test Set)

1. Test Seti Neden Gerekli?
RAG sistemi çalışıyor olabilir ama gerçekten doğru mu çalışıyor?

Öğretmen sınav yapacak ama elinde soru yok → başarıyı ölçemez.
Test Set → cevabı bilinen soru, doğru cevap ve beklenen bilgileri içeren veri kümesi.
Test seti yok → sistem objektif değerlendirilemez.

2. Test Kaydının İçeriği
Her kayıtta 4 alan var: question, reference_answer, keywords, category.

Question: Who won the IIOTY award in 2023?
Reference Answer: Maxine Thompson won the award.
Keywords: Maxine, Thompson, IIOTY
Category: direct_fact
Bu 4 alan birlikte → sistemin cevabını objektif karşılaştırmaya izin verir.

3. Neden JSON Değil JSONL?
JSON: tüm kayıtlar tek liste içinde `[{...},{...}]`.
JSONL: her satır bağımsız bir JSON nesnesi.

{"question":"...","answer":"..."}

{"question":"...","answer":"..."}
Satırlar bağımsız → tek tek okunabilir, bellek verimli kullanılır, yeni test eklemek kolaylaşır, log dosyası gibi satır satır işlenir.

4. Neden Pydantic Kullanıldı?
Gelen veri doğru tipte mi, kontrol etmek gerekir mi?

```
class TestQuestion(BaseModel):
    question: str
    keywords: list[str]
    reference_answer: str
    category: str
```

`question = 123` gibi yanlış veri girilirse → Pydantic hata verir, çünkü `question` string olmalı.
Binlerce test var → yanlış veri değerlendirmeyi tamamen bozabilir → önce doğrulama şart.

5. Doğrulama Akışı
JSONL dosyası okunur → Pydantic modeline uyup uymadığı kontrol edilir.
JSONL → Pydantic → Doğru mu?
├── Evet → Kullan
└── Hayır → Hata Ver

6. Neden 150 Soru?
Tek soru ile evaluation yapılabilir mi?
"CEO kim?" sorusu doğru çıktı → bu sistemin genel olarak iyi olduğu anlamına gelmez, şans eseri olabilir.
Çok sayıda farklı soru → daha güvenilir sonuç.

7. Neden Kategorilere Ayrıldı?
Ortalama başarı yüksek görünse de her kategoride aynı mı?
%90 genel başarı → detaya bakılınca karşılaştırmalı sorularda sistem kötü çıkabilir.
Kategoriye bakmamak → zayıf noktalar ortalamanın altında gizli kalır.

8. Kategori Türleri

- Direct Fact → tek bilgi ister. "CEO kim?"
- Temporal → zamana bağlıdır. "2023'te ödülü kim aldı?"
- Comparative → karşılaştırma ister. "A mı pahalı B mi?"
- Numerical → sayı ister. "Kaç çalışan var?"
- Relationship → ilişki sorar. "CEO hangi departmanı yönetiyor?"
- Spanning → cevap için birden fazla belge gerekir.
- Holistic → tüm belgelerin genelini değerlendirir. "Şirket kültürünü özetle."

Kategori çeşitliliği → sistemin hangi soru tipinde zayıf kaldığını gösterir.

9. AI Engineer Yorumu
Kategori kullanmanın amacı nedir?
Ortalamaya bakmak → genel görünüm verir ama yanıltıcı olabilir.
Kategoriye bakmak → problemin tam olarak nerede olduğunu gösterir.

2. Bölüm

Retrieval Evaluation (Retriever Değerlendirme)

2.1 Akış
Test seti hazır, şimdi ne test ediliyor?
Bu bölümde henüz GPT değerlendirilmiyor, sadece Retriever değerlendiriliyor.

Soru
  |
  ▼
Retriever
  |
  ▼
Top-K Chunk
  |
  ▼
Evaluation
Retriever ayrı test ediliyor → çünkü sistemde bir hata olursa, hatanın retriever'dan mı yoksa GPT'nin cevap üretiminden mi geldiği ayrıştırılabilir. İkisini birlikte test edersek hangi bileşenin bozuk olduğunu anlayamayız.

2.2 MRR (Mean Reciprocal Rank)
En kolay metrik. Sadece şuna bakar: ilk doğru belge kaçıncı sırada geldi?

Örnek 1:
1) CEO ✅
2) Ürün
3) Maaş
→ ilk sırada bulundu → MRR = 1/1 = 1

Örnek 2:
1) Ürün
2) Maaş
3) CEO ✅
→ üçüncü sırada bulundu → MRR = 1/3 = 0.333
Formül: **MRR = 1 / Rank**
Ezber: MRR = ilk doğru sonucun sırası.
Doğru belge üst sıralara çıktıkça → MRR değeri 1'e yaklaşır, geriye düştükçe → değer küçülür.

2.3 nDCG (normalized Discounted Cumulative Gain)
MRR sadece ilk doğru belgeye bakıyordu, peki diğer doğru belgeler ne olacak?
İşte nDCG bunu çözüyor — tüm doğru belgelerin sıralamasını birlikte değerlendiriyor.

İyi sıralama:
1) CEO ✅
2) CEO Yardımcısı ✅
3) Ürün

Kötü sıralama:
1) Ürün
2) Maaş
3) CEO ✅
4) CEO Yardımcısı ✅
İkinci örnekte doğru belgeler alta düşmüş → sıralama kalitesi kötü.
Formül önce DCG hesaplanır:
DCG = Σ (Rel_i / log2(i+1))
Sonra ideal sıralamaya bölünür:
nDCG = DCG / IDCG
Ezber: MRR → sadece ilk doğru belge, nDCG → bütün doğru belgelerin sıralaması.
Doğru belgeler üst sıralarda kümelendikçe → nDCG 1'e yaklaşır.

2.4 Recall@K
Şunu soruyor: doğru belge ilk K sonucun içinde mi bulunmuş?

Top-5 içinde bulundu:
1 2 3 4 5✅ → başarılı

Top-5 dışında kaldı:
1 2 3 4 5 6✅ → başarısız, çünkü Top-5'in dışında
Formül: **Recall@K = Bulunan doğru belge / Toplam doğru belge**
Ezber: Recall = kaç tanesini yakaladın?
K değeri büyüdükçe → doğru belgeyi yakalama şansı artar, dolayısıyla Recall de yükselir.

2.5 Precision@K
Recall "kaç doğruyu buldun" diye soruyordu, Precision ise "bulduklarının ne kadarı doğru" diye soruyor.

1✅ 2✅ 3❌ 4❌ 5❌
→ Precision = 2/5 = 0.40

1✅ 2✅ 3✅ 4✅ 5❌
→ Precision = 4/5 = 0.80
Formül: **Precision@K = Doğru sonuç / İlk K sonuç**
Recall ile Precision farkı:

- Recall → doğruların kaçını buldun?
- Precision → bulduklarının kaçı doğru?
  Getirilen sonuç sayısı (K) arttıkça → Recall genelde yükselir ama Precision genelde düşer, çünkü daha fazla alakasız sonuç da araya karışır.

2.6 Keyword Coverage
Bu projede Recall yerine daha pratik bir metrik kullanılmış: anahtar kelimelerin ne kadarı retriever tarafından bulunmuş.

Keywords: Maxine, Thompson, IIOTY
Retriever'ın bulduğu: Maxine, IIOTY
→ Coverage = 2/3 = %66
Formül: **Coverage = (Bulunan Keyword / Toplam Keyword) × 100**
Bu metrik Recall'a göre daha basit hesaplanıyor → çünkü belge bazında değil, doğrudan kelime bazında eşleşmeye bakıyor. Bulunan anahtar kelime sayısı arttıkça → coverage yüzdesi yükselir, sistemin doğru içeriğe ne kadar ulaştığını gösterir.

3. Bölüm

Answer Evaluation ve RAG Optimization

3.1 Answer Evaluation Akışı
Retriever testi bitti, şimdi ne değerlendiriliyor?
Artık GPT'nin ürettiği cevap değerlendiriliyor, retriever'ın bulduğu belgeler değil.

Retriever

Context

LLM

Cevap

Evaluation
Retrieval evaluation ve answer evaluation ayrı katmanlar → biri diğerinden bağımsız olarak bozulabilir, bu yüzden ikisi ayrı ayrı ölçülür.

3.2 Accuracy
Cevaptaki bilgiler doğru mu?
Yanlış bilgi verilirse → Accuracy düşer.
Bu en temel kontrol → cevabın içeriği gerçeklerle çelişiyorsa diğer metriklerin bir önemi kalmaz.

3.3 Completeness
Cevap eksiksiz mi, her şey anlatılmış mı?

Doğru cevap: Maxine Thompson

GPT'nin cevabı: Maxine
Yanlış değil ama eksik → Completeness düşer.
Doğru bilgi verilse bile bilginin bir kısmı atlanırsa → tam puan alınamaz.

3.4 Relevance
Cevap sorulan soruyla alakalı mı?

Kullanıcı: CEO kim?
GPT: şirketin tarihini anlatmaya başlıyor
Soru ile cevap arasında konu kayması olursa → Relevance düşer.
Cevap ne kadar doğru ve eksiksiz olursa olsun, soruyla ilgisizse bu metrik onu cezalandırır.

3.5 LLM-as-a-Judge
GPT'nin cevabını kim değerlendiriyor?
Bir başka LLM, GPT'nin cevabını puanlıyor.

Question
  |
Reference Answer
  |
Generated Answer
  |
  ▼
Judge LLM
  |
  ▼
Accuracy • Completeness • Relevance • Feedback
İnsan yerine LLM değerlendirici kullanılır → çünkü 150 cevabı elle puanlamak hem yavaş hem tutarsız olur, LLM ise aynı kriterle her cevabı hızlıca puanlar.

3.6 Structured Output
Judge LLM cevabı nasıl döndürüyor?
Serbest metin değil, sabit formatlı JSON döndürüyor.

json
```
{
 "accuracy": 4,
 "completeness": 5,
 "relevance": 5,
 "feedback": "..."
}
```
Format sabitlenirse → çıktı program tarafından otomatik işlenebilir hale gelir, serbest metin olsaydı her seferinde farklı biçimde gelip parse etmek zorlaşırdı.

3.7 Pydantic Doğrulaması (AnswerEval)
Judge LLM'in çıktısı güvenilir mi, kontrol ediliyor mu?
`AnswerEval` modeli devreye giriyor ve şunları kontrol ediyor:

- `accuracy` gerçekten sayı mı?
- `feedback` string mi?
- eksik alan var mı?
  Her şey doğruysa → Python nesnesine çevriliyor.
  Judge LLM'in çıktısı da tıpkı test setindeki gibi doğrulanır → çünkü LLM çıktısı bazen bozuk JSON üretebilir, bu da tüm evaluation'ı çökertebilir.

3.8 Kodların Mantığı

Proje hangi dosyalara bölünmüş?

- `tests.jsonl` → 150 test sorusunun durduğu yer.
- `test.py` → JSONL dosyasını okur → Pydantic ile doğrular → listeye çevirir.
- `eval.py` → tüm evaluation mantığı burada: Retrieval, MRR, nDCG, Coverage, Judge LLM.
- `evaluator.py` → Gradio Dashboard'u yönetir, butona basılınca evaluation başlatır.
  Her dosya tek bir görevden sorumlu → bu sayede kodun hangi kısmının neyi test ettiği kolayca izlenebilir.

3.9 Dashboard

Dashboard'da ne gösteriliyor?

- Retrieval → MRR, nDCG, Coverage
- Answer → Accuracy, Completeness, Relevance
- Ayrıca kategori bazlı grafikler de oluşturuluyor.
  Sonuçlar kategoriye göre görselleştirilir → hangi soru tipinde sistemin zayıf kaldığı tek bakışta görülür.

3.10 RAG Optimization

Metrikler elde edildikten sonra ne yapılıyor?
Ayarlar değiştirilip sistem tekrar test ediliyor.

- **Chunk Size:** 1000 → 500 → 1667 hepsi denenmiş, en iyi sonuç 1000'de alınmış.
- **Top-K:** Chunk küçüldükçe Top-K artırılıyor, çünkü küçük chunk'lar daha fazla belge gerektiriyor.
- **Markdown Splitter:** Denenmiş ama `RecursiveCharacterTextSplitter` daha başarılı çıkmış.
- **Embedding Modeli:** MiniLM → OpenAI Small → OpenAI Large sırasıyla denenmiş.
  Her parametre tek tek değiştirilip test ediliyor → hangi değişikliğin performansı nasıl etkilediği net şekilde ölçülüyor.

3.11 Sonuçlar

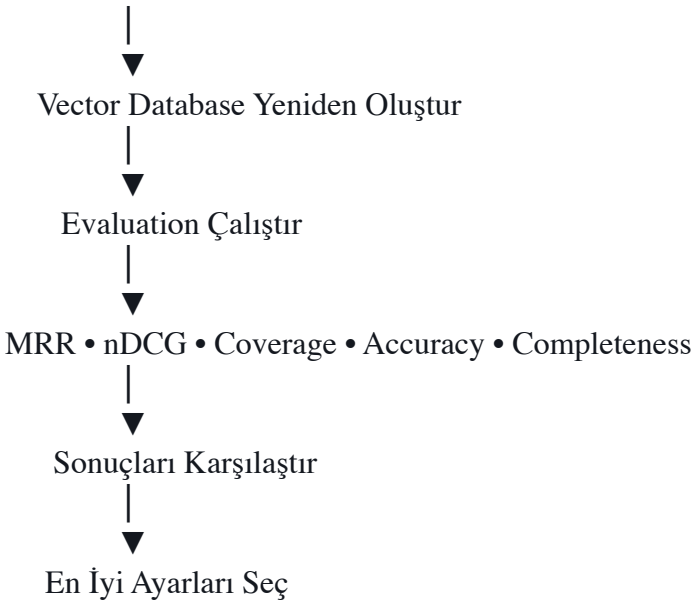
Ayar değişiklikleri metrikleri nasıl etkilemiş?

MRR: 0.72 → 0.78 → 0.79
Coverage: 83% → 92%
Accuracy: 3.99 → 4.21
Embedding modeli güçlendikçe → hem retrieval hem answer metrikleri birlikte yükselmiş.

3.12 En Önemli Zincir

Bütün optimizasyon süreci tek bir akışla özetlenebilir mi?

Chunk Size / Top-K / Splitter / Embedding



Bu döngü her parametre değişikliğinde tekrar ediliyor → RAG optimizasyonu tek seferlik değil, sürekli tekrar eden bir deney döngüsü.

3.13 Günün Çıkarımı
Bu bölümün özeti nedir?
Artık RAG geliştirmek değil, RAG mühendisliği yapmak öğrenildi.
Sistem kurulduktan sonra "çalışıyor" demek yeterli değil → test setleri hazırlanır, Retrieval ve Answer Evaluation metrikleriyle performans ölçülür, farklı chunk boyutu / Top-K / splitter / embedding kontrollü deneylerle karşılaştırılır.
Sezgi yerine ölçülebilir veri kullanılır → böylece en iyi RAG sistemi rastgele değil, kanıta dayalı seçilir.

4.Bölüm
Advanced RAG Techniques

Klasik RAG çalışıyor, peki daha iyi hale getirilebilir mi?

Belgeler → Chunk → Embedding → Vector Database → Retriever → LLM → Cevap
Hybrid RAG, Graph RAG, Agentic RAG, Query Expansion, Re-ranking gibi teknikler sihirli çözüm değil, sadece denenebilecek yöntemler.
Hangi tekniğin işe yaradığı Evaluation ile ölçülmeden karar verilirse → sistem gelişmek yerine rastgele değişir.

4.2 Chunking Research
Belgeleri farklı boyutlarda parçalamak sonucu değiştirir mi?

1000 karakter → 500 karakter → 1667 karakter
RecursiveCharacterTextSplitter / Markdown Splitter
Küçük chunk her zaman iyi değildir, büyük chunk da her zaman iyi değildir → hangisi iyi, Evaluation ile anlaşılır.

4.3 Embedding Research
Embedding modeli Retrieval kalitesini nasıl etkiler?

Kötü embedding → Kötü Retrieval → Kötü Context → Kötü LLM cevabı
MiniLM, OpenAI Small, OpenAI Large karşılaştırılır → embedding modeli zayıfsa, sonraki tüm adımlar da zayıf kalır.

4.4 Prompt İyileştirme
Herkes Advanced RAG'e koşuyor, peki prompt'a bakan var mı?

Bugünün tarihi...
Yalnızca verilen context'i kullan.
Emin değilsen bilmiyorum de.
Retriever doğru context getiriyorsa → iyi bir prompt cevap kalitesini ciddi şekilde artırır.

4.5 Document Preprocessing
Ham veriler embedding modeli için her zaman anlamlı mı?

Ankara / İstanbul / 1500
↓ LLM ile yeniden yazılır
"Ankara ile İstanbul arasındaki bilet fiyatı 1500 TL'dir."
Belge önce düzenlenip sonra embedding'e verilirse → embedding modeli metni çok daha iyi anlar.

4.6 Query Rewriting
Belirsiz bir soru Retriever'ı yanıltır mı?

"Peki geçen yılki?"
↓ LLM yeniden yazar
"2024 yılında IIOTY ödülünü kim kazandı?"
Soru netleştirildikten sonra arama yapılırsa → Retriever çok daha başarılı sonuç bulur.

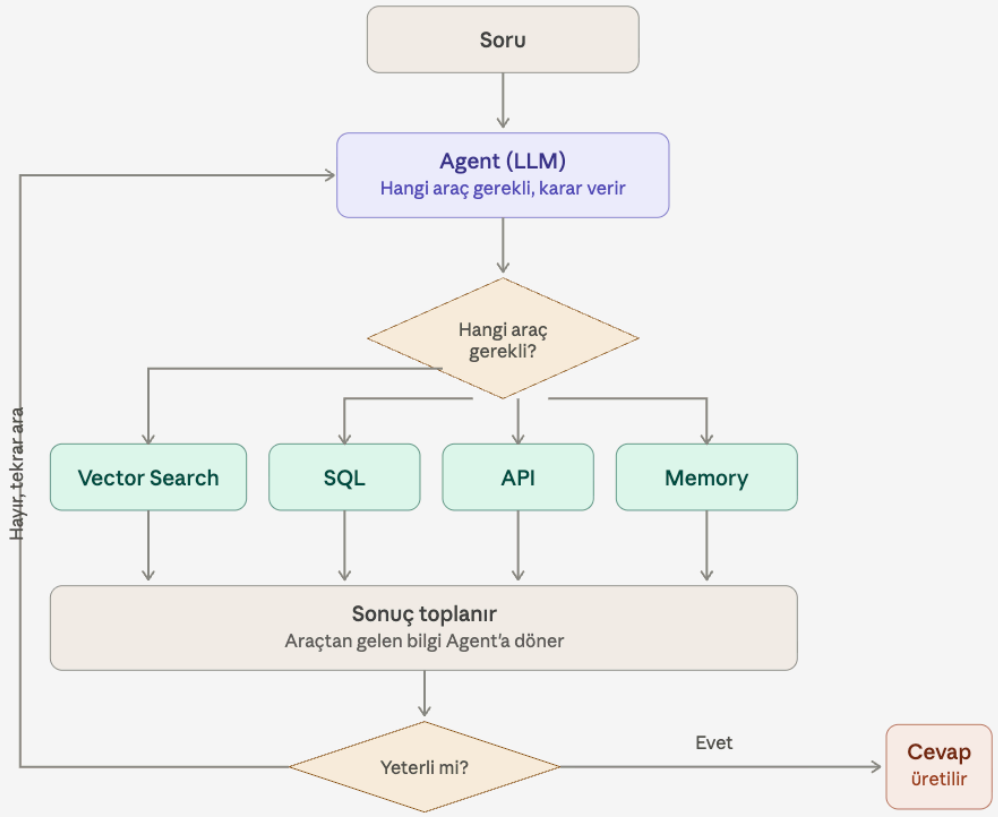
4.7 Query Expansion
Tek bir sorgu yeterli mi, yoksa kaçan belgeler olur mu?

"2023 IIOTY Winner"
↓ LLM
1) IIOTY Winner
2) Innovation Award Winner
3) 2023 Insurellm Award
Birden fazla sorgu birden aranırsa → normalde kaçabilecek belgeler de yakalanır.

4.8 Re-Ranking
Query Expansion sonrası gelen çok sayıda chunk doğrudan LLM'e gönderilebilir mi?

15 chunk → LLM "önem sırasına göre diz" → en alakalı üstte, en alakasız elenir

Agentic RAG'i şemayla netleştirelim — klasik RAG'de akışı biz kontrol ediyorduk, burada LLM kendisi karar veriyor.



4.11 Agentic RAG
Retrieval'ı hep biz mi yönetmeliyiz?
Klasik RAG'de akış sabitti: Retriever → LLM. Agentic RAG'de bu sabit sıra kırılıyor — LLM'e birden fazla araç veriliyor (Vector Search, SQL, API, Memory) ve hangisini ne zaman kullanacağına kendisi karar veriyor.

Şemadaki döngü önemli: Agent bir araç çağırıp sonucu aldıktan sonra "bu yeterli mi?" diye kendine soruyor. Yeterli değilse tekrar arama yapıyor, yeterliyse cevabı üretiyor. Yani klasik RAG'de tek geçişlik bir akış varken, Agentic RAG'de LLM gerektiği kadar döngüye giriyor.

Bu döngü sistemi güçlü kılar → daha karmaşık soruları çok adımlı arama yaparak çözebilir. Ama aynı döngü öngörülebilirliği azaltır → her seferinde aynı sayıda adım atmayabilir, aynı aracı seçmeyebilir.

Chunk'lar önem sırasına dizilip elendikten sonra LLM'e gönderilirse → hem maliyet düşer hem cevap kalitesi artar.

4.9 Hierarchical RAG
Normal RAG çok geniş kapsamlı sorularda yeterli mi?

"Maaşı 50.000 TL'nin altında kaç çalışan var?"
↓
Çalışan Özeti → Departman Özeti → Detay Belgeler
Önce özet belgeler taranıp sonra detaya inilirse → dağınık bilgi tek belgede aranmak zorunda kalmadan toparlanır.

4.10 Graph RAG
Belgeler birbirinden tamamen bağımsız mı ele alınmalı?

CEO → Departman → Çalışan → Proje
Belgeler arasındaki ilişki grafiğe taşınırsa → CEO bulunduğunda ilişkili belgeler de otomatik gelir; ancak çoğu projede aynı sonuç metadata ile de elde edilebilir.

4.11 Agentic RAG
Retrieval'ı hep biz mi yönetmeliyiz?

Eskiden: Retriever → LLM
Şimdi: LLM'e araç veriliyor → Vector Search / SQL / API / Memory
LLM hangi aracı kullanacağına kendisi karar verirse → sistem daha güçlü ama daha az öngörülebilir hale gelir.

4.12 RAG Öldü mü?
Context window büyüdü, Agentic RAG çıktı — RAG'e gerek kaldı mı?
Retriever kullanılsın ya da Agent kullanılsın, amaç aynı: dışarıdan bilgi getirip LLM'in cevabını güçlendirmek.
Amaç dışarıdan bilgi getirmek olduğu sürece → bu hâlâ Retrieval-Augmented Generation'dır, sadece yöntem değişmiştir.

4.13 AI Engineer Bakış Açısı
Bu bölümün asıl dersi teknikler mi?
Hayır — asıl ders düşünme şekli.
Önce problem belirlenir → sonra uygun teknik seçilir → Evaluation yapılır (MRR, nDCG, Accuracy) → gerçekten iyileşme varsa kullanılır, yoksa vazgeçilir.
Teknik seçimi Evaluation'a dayandırılırsa → sistem sezgiyle değil, kanıtla geliştirilir.

5.Bölüm Advanced RAG Laboratuvarı

5.1 Amaç
Bu laboratuvarda RAG, LangChain olmadan mı kuruldu?
Evet — belge yükleme, chunking, embedding, Vector DB, Retrieval, Re-Ranking ve Query Rewriting adımlarının tamamı kendi kodumuzla yazıldı.
LangChain aradan çıkarılınca → RAG'in arka planda gerçekte ne yaptığı daha net görülür.

5.2 Belgeleri Yükleme
Knowledge Base'deki belgeler nasıl okunuyor?

Markdown dosyaları → tek tek okunur
Her belge için: tür, dosya yolu, metin belleğe alınır
Toplam ~76 belge yüklendi.
Belgeler önce belleğe alınırsa → sonraki chunking adımı üzerinde çalışacak veriye sahip olur.

5.3 Semantic Chunking
Chunk'lar bu kez karaktere göre mi bölünüyor?
Hayır — bölme işini LLM yapıyor.

LLM'e görev: "Bu belgeyi soru cevaplama için anlamlı parçalara ayır"
Her chunk içeriği: başlık + kısa özet + orijinal metin
Chunk'a özet ve başlık eklenirse → embedding modeli sadece ham metni değil, parçanın ne anlattığını da anlar.
Çıktı Pydantic Structured Output ile doğrulanıyor → LLM sabit formatta cevap vermek zorunda kalıyor.
Sonuç: ~380–400 semantic chunk.

5.4 Embedding ve Chroma'ya Kaydetme
Chunk'lar vektöre nasıl dönüşüyor?

Chunk → OpenAI Embedding modeli → vektör
vektör + metin + metadata → Chroma Vector Database
LangChain yerine doğrudan OpenAI API + Chroma kullanılırsa → her adım elle kontrol edilebilir hale gelir.

5.5 Vektörleri Görselleştirme
1536 boyutlu bir vektör nasıl görülür?

1536 boyut → t-SNE → 2 boyut → 3 boyut → grafik
Grafikte ürünler, çalışanlar, sözleşmeler ayrı kümeler halinde toplanıyor.
Benzer belgeler embedding uzayında yakın konumlanırsa → embedding modelinin anlamı doğru yakaladığı doğrulanır.

5.6 Re-Ranking
Chroma'nın getirdiği ilk sonuç her zaman en doğru mu?
Hayır.

Chroma → benzerlik skoruna göre chunk'lar getirir
İkinci LLM → chunk'ları soruyla ilgisine göre yeniden sıralar
Örnek: "Manchester University'ye kim gitti?" sorusunda doğru belge ilk sırada değilken, Re-Ranking sonrası ilk sıraya taşınıyor.
Yeni bilgi eklenmez, sadece doğru bilgi öne çıkarılırsa → cevap kalitesi artar.

5.7 Query Rewriting
Kullanıcının sorusu direkt Vector DB'ye mi gidiyor?
Hayır, önce LLM soruyu yeniden yazıyor.

"Who won the IIOTY award?" → "Who received the IIOTY award?"
Kısa/belirsiz sorular yeniden yazılırsa → Retrieval başarısı artabilir.
Ama dikkat: LLM sorguya gereksiz kelime veya şirket adı eklerse → alakasız belgeler gelebilir.
Bu yüzden Query Rewriting her zaman iyileştirmez → mutlaka Evaluation ile test edilmeli.

5.8 Final RAG Pipeline
Tüm parçalar nasıl birleşiyor?

Kullanıcı Sorusu
↓
Query Rewriting
↓
Question Embedding
↓
Chroma Retrieval
↓
Top-K Chunk
↓
Re-Ranking
↓
En Alakalı Context
↓
System Prompt
↓
LLM
↓
Final Cevap
Her adım kendi kodumuzla yazıldığı için → LangChain'e bağımlı olmayan, tamamen kontrol edilebilir bir pipeline ortaya çıkıyor.

5.9 AI Engineer Bakış Açısı
Bu laboratuvarın asıl amacı yeni kod yazmak mıydı?
Hayır — asıl amaç framework'lerin arkasında ne olduğunu öğrenmekti.
Semantic Chunking, Re-Ranking, Query Rewriting eklenirse → Retrieval kalitesi artar, ama hiçbiri tek başına en iyi çözüm değildir → her teknik Evaluation ile test edilmeli.

6. Bölüm

RAG Sistemini Deneylerle Optimize Etme ve Değerlendirme

6.1 Genel Akış
Ayar değiştirdikten sonra ne yapılıyor?

Bir Ayarı Değiştir
↓
ingest.py Çalıştır
↓
Belgeleri Yeniden Parçala
↓
Yeni Embedding Vektörleri Üret
↓
Vector Database'i Yeniden Oluştur

↓
Evaluation Çalıştır
↓
Sonuçları Karşılaştır
Her ayar değişikliğinden sonra bu döngü tekrarlanırsa → değişikliğin gerçek etkisi ölçülebilir hale gelir.

6.2 ingest.py
Bu dosya ne işe yarıyor?

Belgeler Okunur → Chunk'lara Bölünür → Embedding Modeline Gönderilir → Vektör Üretilir → Vector Database'e Kaydedilir
Chunk size, splitter veya embedding modeli değişirse → eski vektörler yeni ayarlarla uyuşmaz → bu yüzden her değişiklikte ingest.py yeniden çalıştırılır.

6.3 Chunk Size ve Top-K Deneyleri
Hangi chunk boyutu daha iyi sonuç veriyor?

Chunk Size = 1000, Top-K = 5 → toplam context ~5000 karakter
Chunk Size = 500, Top-K = 10 → toplam context ~5000 karakter (daha başarılı)
Chunk Size = 1667, Top-K = 3 → toplam context ~5000 karakter (500'den kötü)
Toplam context sabit tutulup sadece chunk boyutu değiştirilirse → hangi boyutun Retrieval'a daha uygun olduğu net görülür.
Çıkarım: chunk çok küçükse bilgi bütünlüğü kaybolur, çok büyükse gereksiz bilgi gelir → en iyi boyut Evaluation ile bulunur.

6.4 Text Splitter Deneyi
Markdown'a özel splitter daha mı iyi çalıştı?
Hayır — bazı bölümleri çok büyük chunk haline getirdiği için Retrieval performansı düştü.

En iyi sonuç: RecursiveCharacterTextSplitter + Chunk Size = 500
Bu sonuç genel bir kural değil, sadece bu veri setine özgü → farklı veri setinde Markdown splitter daha iyi çıkabilir.

6.5 Embedding Modeli Deneyleri
Hangi embedding modeli seçildi?

all-MiniLM-L6-v2 (Hugging Face)
↓
text-embedding-3-small (OpenAI) → ~1500 boyut → MRR = 0.7849
↓
text-embedding-3-large (OpenAI) → ~3000 boyut → MRR = 0.7903
Model değiştikçe ingest.py yeniden çalıştırılıyor → çünkü tüm chunk'ların yeni modelle tekrar vektöre çevrilmesi gerekiyor.
Önemli kural: belgeler hangi modelle vektörleştiyse, soru da aynı modelle vektörleşmeli → aksi halde soru ve belgeler aynı vektör uzayında olmaz, karşılaştırma bozulur.

6.6 Embedding'in Cevap Kalitesine Etkisi
Daha iyi embedding sadece Retriever'ı mı iyileştiriyor?
Hayır, LLM'in cevabını da iyileştiriyor.

MRR: 0.7298 → 0.7903
Keyword Coverage: %83.8 → %92.5
Accuracy: 3.99 → 4.21
Completeness: 3.85 → 4.05
Relevance: 4.57 → 4.71
Zincir şu şekilde işliyor:

Daha İyi Embedding → Daha Doğru Chunk'lar → Daha Kaliteli Context → LLM'e Daha Doğru Bilgi → Daha Kaliteli Cevap
LLM modeli aynı kalsa bile context iyileşirse → cevap kalitesi de yükselir.

6.7 RAG Evaluation
Sistem hangi iki başlıkta değerlendiriliyor?

RAG Evaluation
├── Retrieval Evaluation → Retriever doğru bilgiyi buluyor mu?
└── Answer Evaluation → LLM kaliteli cevap üretiyor mu?
İki katman ayrı ölçülürse → hatanın Retriever'dan mı LLM'den mi geldiği net ayrışır.

6.8 Retrieval Evaluation Sonuçları
150 test sorusuyla ölçülen sonuçlar neler?

MRR (Mean Reciprocal Rank): 0.8779 → doğru chunk çoğunlukla ilk sıralarda
nDCG: 0.8479 → sonuçlar genel olarak doğru önem sırasında
Keyword Coverage: %94.6 → gerekli bilgilerin neredeyse tamamı bulunmuş
150 farklı soru üzerinden ölçülürse → sonuç tek soruya göre şansa bağlı olmaktan çıkar, güvenilir hale gelir.

6.9 Kategori Bazlı Sonuçlar
Hangi soru tipinde Retriever zorlanıyor?

Başarılı: Relationship, Direct Fact, Numerical
En zor: Spanning → cevap tek chunk'ta değil, birden fazla belgede
Zor: Holistic → belgenin genelini anlamayı gerektiriyor
Kategoriye göre sonuç incelenirse → genel ortalamanın arkasındaki zayıf nokta görülür.

6.10 Answer Evaluation
LLM'in cevabı nasıl puanlanıyor?

Test Sorusu → Retriever Chunk Bulur → LLM Cevap Üretir → Cevap Referansla Karşılaştırılır → LLM-as-a-Judge → Puanlama

Hakem LLM üç ölçütle puanlıyor:

- **Accuracy** (1-5): bilgiler doğru mu?
- **Completeness** (1-5): gerekli bilgilerin tamamı var mı?
- **Relevance** (1-5): cevap soruyla ilgili mi?
  Ayrıca kısa bir **feedback** üretiliyor, örn. "Cevap doğru ancak fiyat bilgisi eksik."
  Puanın yanında feedback de üretilirse → sadece kaç aldığı değil, neden o puanı aldığı da anlaşılır.
  150 test sırayla çalıştırılıyor (Run Evaluation) → ilerleme yüzdesi ve geçen süre canlı gösteriliyor.

6.11 Genel Sonuç ve AI Engineer Notu

Bu veri setinde en iyi kombinasyon ne oldu?

RecursiveCharacterTextSplitter + Chunk Size = 500 + Top-K = 10 + text-embedding-3-large
Genel optimizasyon döngüsü:

Ayarı Değiştir → ingest.py Çalıştır → Vector DB Yeniden Oluştur → Evaluation Çalıştır → Metrikleri Karşılaştır → En İyi Ayarı Seç
AI Engineer'ın gerçek görevi:

Sistemi Kurmak → Test Etmek → Metriklerle Ölçmek → Ayarları Değiştirmek → Yeniden Test Etmek → En İyi Yapıyı Seçmek
Retriever doğru bilgiyi bulsa bile LLM'in bu bilgiyi doğru, eksiksiz ve alakalı işlemesi gerekir → yalnızca cevap üretmek sistemi başarılı yapmaz, ikisi birlikte ölçülmeli.

